Library Management System

Full architecture: users, frontend, API, database, deployment and data flow

Users

Anyone with the link

Customer
Any username, role Customer

Admin
Username and password check in the browser

↕ opens the Render URL in a browser

Frontend

HTML · CSS · vanilla JS

index.html
Login, Browse, My Books, Admin panel

styles.css
Layout and theme

script.js
Login logic, calls fetch to /api, 7-day borrow period

Browse books
See copies available

Borrow and return
My Books list

Suggest a book
Sent to admin

Admin tools
Add, edit, delete, approve

↕ HTTPS · JSON · /api/*

Render web service · Node.js 20+ · free plan · PORT from env

Backend: Express 5

server.js

Middleware
cors, express.json, static public/

Books routes
GET, POST, PUT, DELETE /api/books

Borrow routes
POST /borrow, /return; GET /borrows/:user, /admin/borrows

Suggestion routes
GET, POST, DELETE, approve

Download route
GET /api/admin/download-db?key=ADMIN_KEY

prepared SQL statements

Data layer

database.js · better-sqlite3

library.db (SQLite file)

Created on startup if tables are missing. Seeded with 3 books if empty. DB_PATH env var can point to a persistent disk.

books

id (PK)	title, author, quantity

borrows

id (PK)	book_id (FK), username, borrow_date, returned

suggestions

id (PK)	title, author, suggested_by

deploys on every git push

Deployment pipeline

DevOps

Local PC
VS Code, DB Browser, Git

→

GitHub
masanebhumika-alt / Library-Management-Sysytem

→

Render build
npm install

→

Render start
npm start, live URL

inspect live data

Inspecting the live database

Secret key
ADMIN_KEY set in Render, Environment

→

library-live.db
Downloaded from the browser

→

DB Browser
Browse Data tab

⚠ **Data flow is one way.** Your pushed `library.db` is the starting data. Changes made on the live site stay on Render and reset on every restart or redeploy on the free plan. For permanent data, add a Render disk with `DB_PATH=/var/data/library.db`, or move to Postgres.